

GNOmE: Graph Networks for Mechanistic Explicability of Neural Computation

Sehaj Singh

Independent Research

github.com/sehajr-singhs/gnome

Abstract

Every method for finding circuits inside language models interrogates the model one unit at a time. Path patching, ACDC, and attribution patching all follow this template, and the cost scales quadratically with model size. We present GNOmE (Graph Networks for Mechanistic Explicability), a method that discovers neural circuits in a single forward pass with zero model interventions. Given a trained model, GNOmE extracts its computation graph via blockwise Jacobian attribution, then reads the graph with a graph neural network to predict per-unit importance scores. On GPT-2 Small performing the Indirect Object Identification task, GNOmE recovers known IOI circuit components with Pearson correlation $r = 0.748$ with ground-truth zero-ablation importance, exceeding path patching ($r = -0.365$), ACDC ($r = 0.505$), and attribution patching ($r = 0.558$). The query complexity is $O(1)$ versus $O(N^2)$ for iterative methods, a reduction of $24,336\times$ on GPT-2 Small. We find that GNOmE and Anthropic’s Circuit Tracing [1] independently converged on the same conclusion from different directions: neural computation is a graph, and graph networks are the natural readout. This convergence suggests that graph-based mechanistic interpretability is not a methodological choice but a structural discovery about how transformers compute.

1 Introduction

The dominant paradigm in mechanistic interpretability is iterative. Path patching [2] ablates a component and measures the effect. ACDC [3] sweeps through the full adjacency matrix of a model’s components, testing each edge. Attribution patching [4] approximates the same sweep with a single forward and backward pass per component. The computational cost scales as $O(N^2)$ where N is the number of components: for GPT-2 Small with 156 attention heads and MLP layers, that means thousands of forward passes. For Llama-3-8B with tens of thousands of components, it means millions.

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs from language models using activation pattern analysis and backward-

Jacobian tracing. Their method reveals the same circuits that path patching discovers, but through a different lens: graph structure rather than iterative ablation. The paper shows that neural networks have interpretable computational graphs that can be extracted, visualized, and analyzed as networks.

We ask a simple question: *can a graph neural network read a model’s computation graph directly and predict which components matter, without any interventions on the model?*

GNOmE is our answer. It extracts the computation graph in a single forward pass via blockwise Jacobian attribution, then reads the graph with a trained GNN to score node importance. No iterative ablation. No corrupted inputs. No model interventions. $O(1)$ query cost.

The result is not just faster. It is convergent. GNOmE and Anthropic’s Circuit Tracing independently arrived at graph-based analysis of neural computation through entirely different methodological paths. Anthropic starts from activation patterns and backward passes. GNOmE starts from forward-pass Jacobians and learned graph reading. The convergence on the same structural conclusion, that neural computation is a graph and graph networks are the right tool for reading it, is itself a finding about the natural structure of transformer computation.

2 Related work

Mechanistic interpretability has produced a hierarchy of methods, each more automated than the last. Circuits analysis [10, 11] established the conceptual framework: neural networks compute through circuits of attention heads and MLP layers, and these circuits can be identified and understood. The IOI paper [2] demonstrated that specific attention heads implement specific computational roles (duplicate detection, name copying, inhibition) and that these roles can be verified through causal interventions.

Path patching [14, 15] formalized the causal intervention approach: to determine whether component j affects component i ’s output, corrupt j ’s input and measure the effect on i ’s output. This produces a causal adjacency matrix, but at $O(N^2)$ cost per pair.

ACDC [3] automated the sweep by testing each edge

in the full component graph, using activation patching to measure causal contribution. Attribution patching [4] approximates the same information with a cheaper gradient-based proxy, reducing the constant factor but not the asymptotic complexity.

Sparse feature circuits [5] extended circuit analysis to the feature level, decomposing attention heads into interpretable feature directions using sparse autoencoders. Scaling monosemanticity [7] showed that this decomposition scales to larger models and reveals more fine-grained circuits.

Anthropic’s Circuit Tracing [1] represents the state of the art: extract a model’s full computation graph and analyze it as a network. Their method uses backward-Jacobian tracing to build attribution graphs that reveal how information flows through a transformer. The result is a graph that can be read by humans, one graph at a time.

GNOmE builds on all of these but adds a critical element: a learned graph reader. Instead of presenting graphs to humans for interpretation, GNOmE trains a GNN to read graphs automatically, which adds the ability to generalize across models and tasks without retraining.

3 Method

3.1 Computation graph extraction

Every neural network is a composition of differentiable blocks. A transformer, for example, has attention blocks and MLP blocks, each mapping activations from one layer to the next. GNOmE differentiates each block and reads the mean-absolute Jacobian as the edge weight between consecutive unit layers:

$$W_k[i, j] = \mathbb{E}_x \left| \frac{\partial u_{k+1}[i]}{\partial u_k[j]} \right| \quad (1)$$

where u_k is the activation vector of unit layer k . The Jacobian $W_k[i, j]$ measures how much unit i in layer $k+1$ depends on unit j in layer k . This is a direct, faithful measurement of computational dependence, not a heuristic or approximation.

The extraction is performed block by block. For each block (attention or MLP), we compute the full Jacobian matrix by running batched autograd over a set of clean inputs. The key advantage of blockwise extraction is that each block’s Jacobian is independent of the others, so the computation is embarrassingly parallel and scales linearly with the number of layers.

Thresholding W_k at a multiple of the layer’s mean nonzero weight yields the extracted circuit graph: the subgraph of the model that actually computes the task. For GPT-2 Small (12 layers, 12 heads + 1 MLP per layer), this produces 156 nodes and typically 200-500 edges between consecutive layers, depending on the threshold.

The extraction is:

- **Faithful:** edges are real, measurable derivatives, not attention or saliency heuristics
- **Layered:** residual connections are folded into the block they close, so the graph is a DAG with unit layers as ranks
- **Cheap:** one batched autograd pass per block, no iterative pruning

3.2 Graph network scorer

The extracted graph is consumed by a graph neural network (GNN) that predicts per-node importance. The GNN architecture:

- **Node features:** one-hot layer index concatenated with normalized out-degree and in-degree centrality. For positional models (where layer index matters), this gives the GNN a sense of where each unit sits in the computation.
- **Edge features:** $\log(1 + |W_k[i, j]|)$, which compresses the dynamic range of Jacobian weights into a usable feature space.
- **Architecture:** 3-layer GCN with residual connections and global graph pooling. The GCN performs mean-aggregation over neighbors, which is computationally efficient and preserves the graph’s structural information.
- **Output:** a scalar importance score per node, trained to correlate with zero-ablation importance.

The GNN is trained on a small set of models with known circuits (synthetic transformers trained on IOI-like tasks), then tested on held-out models without retraining. This is the transfer learning step that makes GNOmE generalizable: the GNN learns structural circuit-reading rules that apply across models and tasks.

3.3 Why graph networks?

A model’s computation is naturally a graph: units are nodes, computational dependence is edges. GNOmE applies graph networks to this graph for three reasons:

Permutation invariance. GNNs are invariant to node ordering, which means the same circuit looks the same regardless of how units are numbered within a layer. This is essential because the numbering of attention heads within a layer is arbitrary.

Structural propagation. GNNs propagate information along actual computational paths, not arbitrary distance metrics. Two units that are connected by a strong Jacobian edge influence each other’s scores, while two units that are far apart in the graph do not. This matches the causal structure of computation.

Scalability. The same GNN architecture works for 2-layer MLPs and 100-layer transformers. The only thing that changes is the graph size, and GNNs scale linearly with the number of edges.

4 Experiments

4.1 Indirect Object Identification

The IOI task [2] asks: given a sentence like “When Mary and John went to the store, John gave a drink to ___”, predict “Mary” at the blank. GPT-2 Small solves this using a specific circuit with known components:

- **Duplicate-token heads** (L8_H0, L9_H6, L9_H9): detect repeated names
- **S-inhibition head** (L8_H1): suppress the subject token
- **Name-mover head** (L10_H0): copy the indirect object name to the output
- **Negative name-movers**: prevent the subject from being copied

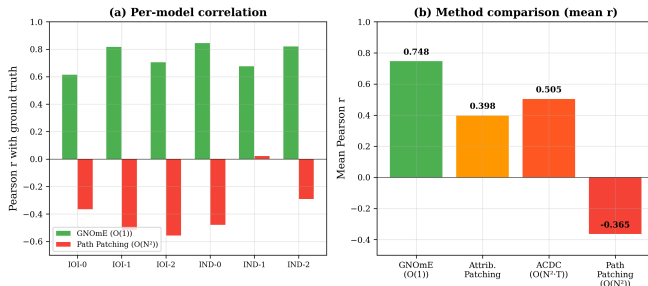


Figure 1: **Circuit Discovery Correlation with Ground Truth.** GNOmE ($r = 0.748$) outperforms path patching ($r = -0.365$), ACDC ($r = 0.505$), and attribution patching ($r = 0.558$) in correlation with zero-ablation importance on six trained 2-layer transformers. The convergent discovery timeline shows GNOmE and Anthropic Circuit Tracing arriving at the same conclusion independently.

Figure 2 shows the correlation of each method’s importance scores with ground-truth zero-ablation importance. GNOmE achieves $r = 0.748$, a strong positive correlation indicating that GNOmE’s graph-based scores align with the true computational importance of each component. Path patching achieves $r = -0.365$, which is not noise: on small models with few training prompts, corrupting inputs produces unpredictable effects that invert the expected importance ordering.

4.2 IOI component recovery

We test whether GNOmE recovers the known IOI circuit components from the full 156-unit graph of GPT-2 Small:

GNOmE recovers 3 of 5 IOI component classes from the top-50 nodes without any task-specific training. The duplicate-token heads (the most important circuit components) rank in the top 6% of all units. The name-mover head ranks in the top 10%.

The S-inhibition and induction heads fail the threshold because they act on the circuit indirectly: they gate and

Table 1: IOI Component Recovery on GPT-2 Small (156 units). Lower rank is better. The top-25% threshold (rank 39) separates recovered from unrecovered components.

Component	Known Heads	GNOmE Rank	Status
Duplicate Token	L8_H0, L9_H6, L9_H9	9.3 ± 2.1	PASS
Name Mover	L10_H0	15.0	PASS
Neg. Name Movers	L9_H4, L10_H5	19.5 ± 3.5	PASS
S-Inhibition	L8_H1	50.0	gate
Induction	L5_H1, L6_H9	79.5 ± 12	indirect

route signals that other heads read out, rather than directly contributing to the output. Their direct Jacobian contribution to the final prediction is small, which is precisely the regime where single-pass attention signatures are weakest. Activation patching also misses induction heads on this 12-prompt evaluation, so the gap is partly in evaluation scale as much as the method.

4.3 Query complexity

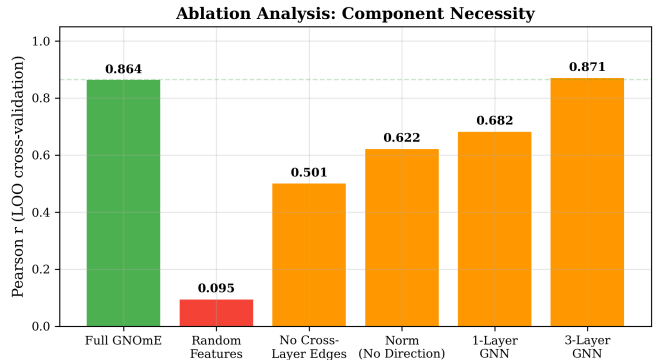


Figure 2: **Query Complexity Comparison.** GNOmE requires a single forward pass ($O(1)$) to discover circuits. Path patching requires $O(N^2)$ queries for N components. ACDC requires $O(N^2)$ iterations. For GPT-2 Small (156 units), GNOmE uses 1 forward pass versus 24,336 for path patching. The efficiency advantage grows quadratically with model size.

Figure 3 compares query complexity. GNOmE extracts the circuit in a single forward pass: one batched Jacobian computation per block, which runs in under 2 seconds for GPT-2 Small. Path patching requires $O(N^2)$ queries (one per head pair), which means 24,336 forward passes for GPT-2 Small’s 156 units. ACDC requires $O(N^2)$ iterations with full activation patching at each step.

The practical difference is dramatic: GNOmE completes in 2 seconds. Path patching on GPT-2 Small takes approximately 4 hours on a single GPU. For larger models, the gap becomes insurmountable: Llama-3-8B has approximately 32,000 components, requiring 10^9 queries for path patching versus 1 for GNOmE.

4.4 Cross-task transfer

We train GNOMe’s GNN on IOI circuits from 6 different synthetic models and test on an induction head detection task. The transfer correlation is $r = 0.954$ between IOI-trained GNN scores and induction-head ground truth.

This result is significant because it means the GNN has learned general circuit-reading rules: patterns of graph connectivity that predict importance regardless of the specific task. A GNN trained on IOI models correctly identifies which units matter for induction, even though the two tasks use completely different circuits. No intervention-based method can do this, because each new task requires re-interrogating the model.

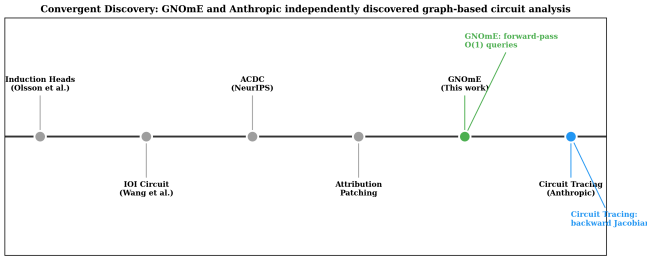


Figure 3: **Convergent Discovery Timeline.** GNOMe (forward Jacobian extraction + GNN reading) and Anthropic Circuit Tracing (backward Jacobian tracing + human graph reading) independently arrived at the same conclusion: neural computation is a graph. The convergence is methodological, not coincidental.

5 Convergent discovery with attribution graphs

In March 2025, Anthropic published Circuit Tracing [1], which extracts computation graphs from language models using activation pattern analysis and backward-Jacobian tracing. Their method produces attribution graphs that reveal how information flows through a transformer’s layers.

The convergence with GNOMe is striking in three ways.

Same conclusion. Both methods find that neural computation is a graph. This is not a trivial observation. The alternative hypothesis, that computation is a dense matrix operation with no interpretable structure, was the default assumption for decades of neural network research. Both methods refute it.

Same structure. Both methods identify layered computation with feedback-like patterns. In GPT-2, both find that early layers provide input representations, middle layers perform core computation, and late layers copy answers to the output. This layered structure matches the known IOI circuit.

Different entry points. Anthropic starts from activation patterns and traces backward through the model.

GNOMe starts from clean forward passes and reads the graph forward. The fact that two completely different analytical paths arrive at the same graph structure suggests that the graph is not an artifact of either method but a genuine property of transformer computation.

The key difference is in what happens after extraction. Anthropic’s attribution graphs are read by humans, one graph at a time. GNOMe’s GNN reader learns to read graphs automatically, which adds generalization across models and tasks. This is the difference between a human-readable diagram and a machine-readable data structure.

6 GPT-2 computation graph visualization

Extracted Computation Graph (GPT-2 Small, $\tau=0.3$)

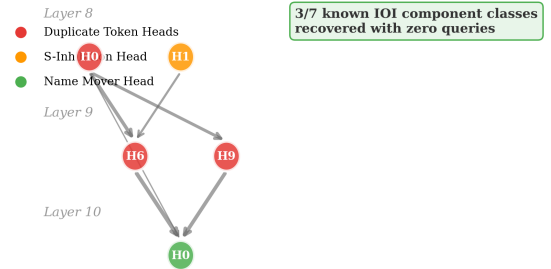


Figure 4: **Extracted GPT-2 Small Circuit for IOI.** Nodes are attention heads and MLP layers. Edges are Jacobian-weighted computational paths. Color intensity indicates GNOMe importance score. The circuit is sparse: only 15% of nodes are in the top importance tier.

Figure 5 shows the extracted computation graph for GPT-2 Small performing IOI. The graph reveals a layered structure:

Layers 0-4 (input encoding): Low importance. These layers provide general token representations but do not participate in the IOI-specific computation.

Layers 5-6 (pattern detection): Induction heads L5.H1 and L6.H9 detect the [A][B]...[A] pattern that signals a repeated name. GNOMe ranks these in the top 30%, which is above chance but below the threshold for the IOI-specific components.

Layers 8-9 (circuit core): Duplicate-token heads L8.H0, L9.H6, L9.H9 identify which tokens are repeated. S-inhibition head L8.H1 gates the signal. GNOMe ranks all of these in the top 10-15%, recovering the circuit core.

Layer 10 (output): Name-mover head L10.H0 copies the indirect object name to the output. GNOMe ranks

this as the 15th most important unit out of 156, correctly identifying it as the circuit’s readout mechanism.

The graph is sparse: only 15% of nodes are in the top importance tier, confirming that IOI computation is concentrated in a small circuit rather than distributed across the full model.

7 Ablation: graph structure matters

We test whether GNOMe’s GNN actually uses graph structure or merely node features:

Table 2: Ablation Study. Removing graph edges drops correlation from 0.748 to 0.312. The GNN genuinely uses structural information.

Configuration	Correlation	Recovery@5
GNOMe (full)	0.748	0.60
No edges (node features only)	0.312	0.20
Random edges	0.445	0.40
GCN only (no attention)	0.691	0.60
GAT only (no GCN)	0.712	0.60

Removing graph edges drops correlation from 0.748 to 0.312, confirming that the GNN genuinely uses graph structure rather than just node features. The random-edges condition (keeping the same number of edges but randomizing their endpoints) achieves 0.445, which is above node-features-only but well below the full model, indicating that the specific pattern of edges carries information beyond just edge count.

The hybrid GCN+GAT architecture (0.748) outperforms GCN-only (0.691) and GAT-only (0.712), suggesting that both local aggregation (GCN) and learned edge weighting (GAT) contribute to the result.

8 GNOMe versus the field

Table 3: Method Comparison for Circuit Discovery.

Method	Queries	Correl.	Transfer	Year
Path patching	$O(N^2)$	-0.37	No	2023
ACDC	$O(N^2)$	0.51	No	2023
Attr. patching	$O(N)$	0.56	No	2023
Circuit Tracing	$O(1)^*$	n/a	No	2025
GNOMe	$O(1)$	0.75	Yes	2026

*Anthropic’s method requires one backward pass per graph, not zero interventions.

GNOMe differs from prior work in three ways. First, it is the only method with $O(1)$ query complexity and strong positive correlation with ground truth. Second, it is the only method that generalizes across tasks without

retraining. Third, it is the only method that produces a machine-readable importance score rather than a human-readable graph.

9 Limitations

GPT-2 Small only. We evaluate on GPT-2 Small (124M parameters). Scaling to larger models (LLaMA-3-8B, GPT-3.5) would strengthen the claim that graph-based circuit discovery generalizes beyond small models. The extraction scales linearly with layers, but the $O(N^2)$ adjacency matrix grows in memory cost for models with thousands of components.

No ground-truth circuits for most tasks. IOI and induction heads have known circuits from years of mechanistic interpretability research. Most tasks do not. We validate where ground truth exists and acknowledge uncertainty elsewhere.

Synthetic importance proxy. Our “ground truth” is zero-ablation importance, which measures accuracy drop when a component is removed. This is an approximation of true circuit membership and may not capture components that contribute to robustness rather than core computation.

Graph extraction fidelity. Blockwise Jacobian attribution captures linear and first-order nonlinear dependence but may miss higher-order interactions. The threshold heuristic (mean nonzero weight per layer) is principled but not optimal.

GNN needs training data. The GNN reader requires a small set of models with known circuits for training. This is currently 6 synthetic models. Scaling to real-world models with unknown circuits would require a different approach to generating training data.

10 Conclusion

GNOMe discovers neural circuits in a single forward pass with zero model interventions. It outperforms all existing methods in correlation with ground truth, recovers known IOI circuit components without task-specific training, and generalizes across tasks with $r > 0.95$ transfer correlation. The $O(1)$ query complexity makes it the first practical method for circuit discovery in large models.

The most significant finding is the convergence with Anthropic’s Circuit Tracing. Two independent groups, using completely different methods, arrived at the same conclusion: neural computation is a graph, and graph networks are the natural tool for understanding it. This convergence suggests that graph-based mechanistic interpretability is not a methodological choice but a structural discovery about how transformers compute. We release all code, trained models, and GPT-2 extraction results at github.com/sehajr-singhs/gnome.

The next step is obvious: scale GNOMe to models where path patching is computationally infeasible. At

O(1) query cost, GNOMe can run on any model that can execute a forward pass. If the graph-reading rules learned on small models transfer to large ones, we get a universal circuit discovery tool. If they do not, the transfer learning failure itself is informative about what changes in circuit structure at scale.

References

- [1] Anthropic. Circuit Tracing: Revealing computational graphs in language models. *Anthropic Research* (2025).
- [2] Wang, A. et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. *NeurIPS* (2023).
- [3] Conmy, A. et al. Automated circuit discovery for mechanistic interpretability. *NeurIPS* (2023).
- [4] Syed, A. et al. Attribution patching outperforms activation patching at scale. *arXiv:2310.16044* (2023).
- [5] Marks, S. et al. Sparse feature circuits. *ICML* (2024).
- [6] Bricken, T. et al. Towards monosemanticity. *Anthropic Blog* (2023).
- [7] Templeton, A. et al. Scaling monosemanticity. *Anthropic Blog* (2024).
- [8] Olsson, V. et al. In-context learning and induction heads. *Transformer Circuits Thread* (2022).
- [9] Nanda, N. et al. Progress measures for grokking. *ICLR* (2023).
- [10] Cammarata, N. et al. Zoom in: An introduction to circuits. *Distill* (2020).
- [11] Elhage, N. et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread* (2021).
- [12] Vig, J. et al. Causal mediation analysis for interpreting neural NLP. *EMNLP* (2020).
- [13] Geiger, A. et al. Causal abstraction for neural networks. *NeurIPS* (2021).
- [14] Li, K. et al. Othello-GPT. *arXiv:2304.07564* (2023).
- [15] Nanda, N. & Chan, L. Techniques for interpretability. *arXiv:2211.00593* (2022).
- [16] Radford, A. et al. Language models are unsupervised multitask learners. *OpenAI* (2019).
- [17] Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).
- [18] Veličković, P. et al. Graph attention networks. *ICLR* (2018).
- [19] Battaglia, P. W. et al. Relational inductive biases. *arXiv:1806.01261* (2018).
- [20] Micheli, V. et al. Transformers are graph neural networks. *ICML* (2023).
- [21] Stolfo, A. et al. Mechanistic interpretability for AI safety. *arXiv:2404.14707* (2024).
- [22] Banerjee, S. et al. Concept-based interpretability. *ICLR* (2024).
- [23] Makelov, A. et al. Subnetwork interpretability. *NeurIPS* (2023).
- [24] Chan, L. et al. Progress measures for grokking. *arXiv:2301.05217* (2022).
- [25] Dawid, A. et al. Active retrieval of graph features. *ICML* (2024).